

Home

Problem

Solution

Architecture

Impact



THREAT PILOT



Agentic SBOM Risk Auditor • CB-02 · Cyber Security + Blockchain • Hackatopia 2K26

CB-02 · Automated SBOM Risk Auditor

A typical Python service pulls in 50–200 open-source packages, most of them transitive. Scanners report every known CVE in every one of them, whether or not the app ever runs the affected code.

Who is affected

Developers buried in alerts, security engineers with no evidence, and leads who cannot ship a fix safely.

Why it matters

Teams triage by CVSS severity alone. Nobody fixes 133 alerts, so nobody fixes any, and the one reachable bug ships.

THE PROBLEM: ALERT FATIGUE

86%

of commercial codebases
contain vulnerable open source

71%

of dependencies are transitive,
invisible in requirements.txt

133 → 1

scanner alerts on a 59-package
repo vs. actually reachable

EXISTING TOOLS & THE GAP

The gap

No tool proves which CVEs your code actually reaches, shows the evidence, and ships a verified fix behind human approval.

Dependabot & pip-audit

List every package with a known CVE and bump versions. They never ask whether your code calls the vulnerable function: all 133 alerts on our test repo.

Snyk & commercial SCA

Reachability exists for some languages as a paid, closed feature. The reasoning is hidden; there is no call chain you can check yourself.

Triage by CVSS score

Teams sort by severity. A 9.8 in a package that is never imported outranks a 7.5 that runs on user input.

Fixing by hand

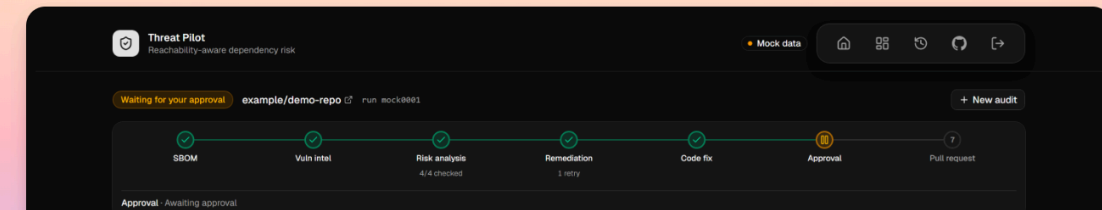
Engineers pick versions, check conflicts and write PRs manually. Blind auto-bumps can break the build or leave the CVE open.

OUR SOLUTION: THREAT PILOT

Threat Pilot proves which vulnerabilities your application can actually reach, ranks them by real risk, and opens a verified pull request that upgrades the dependency and rewrites the affected code, only after a human approves.

Threat Pilot: fix the vulnerabilities your code actually reaches

Scanners flag every CVE in every package. Threat Pilot proves which ones your application can reach, ranks them by real risk, and opens a verified pull request that upgrades the dependency and rewrites the affected code.

[Open your dashboard](#)[Audit a public repository](#)

INPUT

A public GitHub repository

Sign in with GitHub, pick a repo or paste a URL.
Threat Pilot reads its requirements.txt.



PROCESSING

7 LangGraph agents

SBOM → live CVE data → reachability proof →
risk score → verified fix → human gate.



OUTPUT

Evidence + a pull request

Findings ranked by real risk with file:line evidence,
and a verified PR opened after approval.

Severity is not risk

Same repo, two findings, ranked two ways

SCANNER RANKING · BY CVSS

Pillow · L0, never imported

 9.8 ranked #1

PyYAML · L2, reachable

 7.5

THREAT PILOT RANKING · BY RISK SCORE

Pillow · L0, never imported

 2.06

PyYAML · L2, reachable

 14.58 ranked #1

score = CVSS × reach × (1 + EPSS) × KEV × direct

reach weight: L0 0.2 · L1 0.6 · L2 1.0

$7.5 \times 1.0 \times 1.62 \times 1.0 \times 1.2 = 14.58$

INNOVATION & UNIQUE VALUE

USP: the open, explainable auditor that turns hundreds of CVE alerts into one evidence-backed, verified, human-approved pull request.

What is genuinely new

- **Reachability proof** L0, L1 or L2 for every CVE, with file, line and the call chain into the library.
- **A fix that works** smallest safe upgrade plus rewritten functions, checked by a critic loop.
- **Human-gated** nothing touches GitHub until someone approves.

Why teams choose it

- **vs Dependabot / pip-audit** 133 alerts become 1 reachable finding.
- **vs Snyk** open, free data sources, and the evidence is shown, not hidden.
- **Trustworthy AI** the LLM never produces facts; every output is checked in code, and it runs with no model at all.

HOW IT WORKS

User journey

1. Sign in with GitHub and pick a repository.
2. Watch each agent run live over SSE.
3. Read ranked findings with file:line evidence.
4. Review upgrades, rewritten code and PR text.
5. Approve, and the pull request opens.

Decision points

- **Verifier** fix still vulnerable or won't resolve? Retry with a new version, max 2, then report.
- **Nothing to change** no reachable fix needed: end with a report.
- **Human gate** rejected: run ends, GitHub is never touched.



[Home](#)[Problem](#)[Solution](#)[Architecture](#)[Impact](#)

TECHNOLOGY STACK

Frontend · React 18, TypeScript, Vite, Tailwind v4, shadcn/ui

Typed contract mirrors backend schemas; instant reloads for a 24-hour build.

Backend · Python 3.12, FastAPI, Server-Sent Events

Async and Pydantic-typed. Python's own ast parses the code being audited; pip resolves fixes.

AI / agents · LangGraph 1.2, Gemini 3.5 Flash, Ollama

Send() fan-out, cycles and interrupt() built in. CrewAI/AutoGen chat; we need an auditable graph.

Database · Neon Postgres, SQLite fallback

Serverless, with a first-party LangGraph checkpointer, so paused audits survive restarts.

APIs · Syft, OSV.dev, EPSS, CISA KEV, GitHub

Free, no-key, live vulnerability data with fixed-version ranges; LangSmith traces every agent.

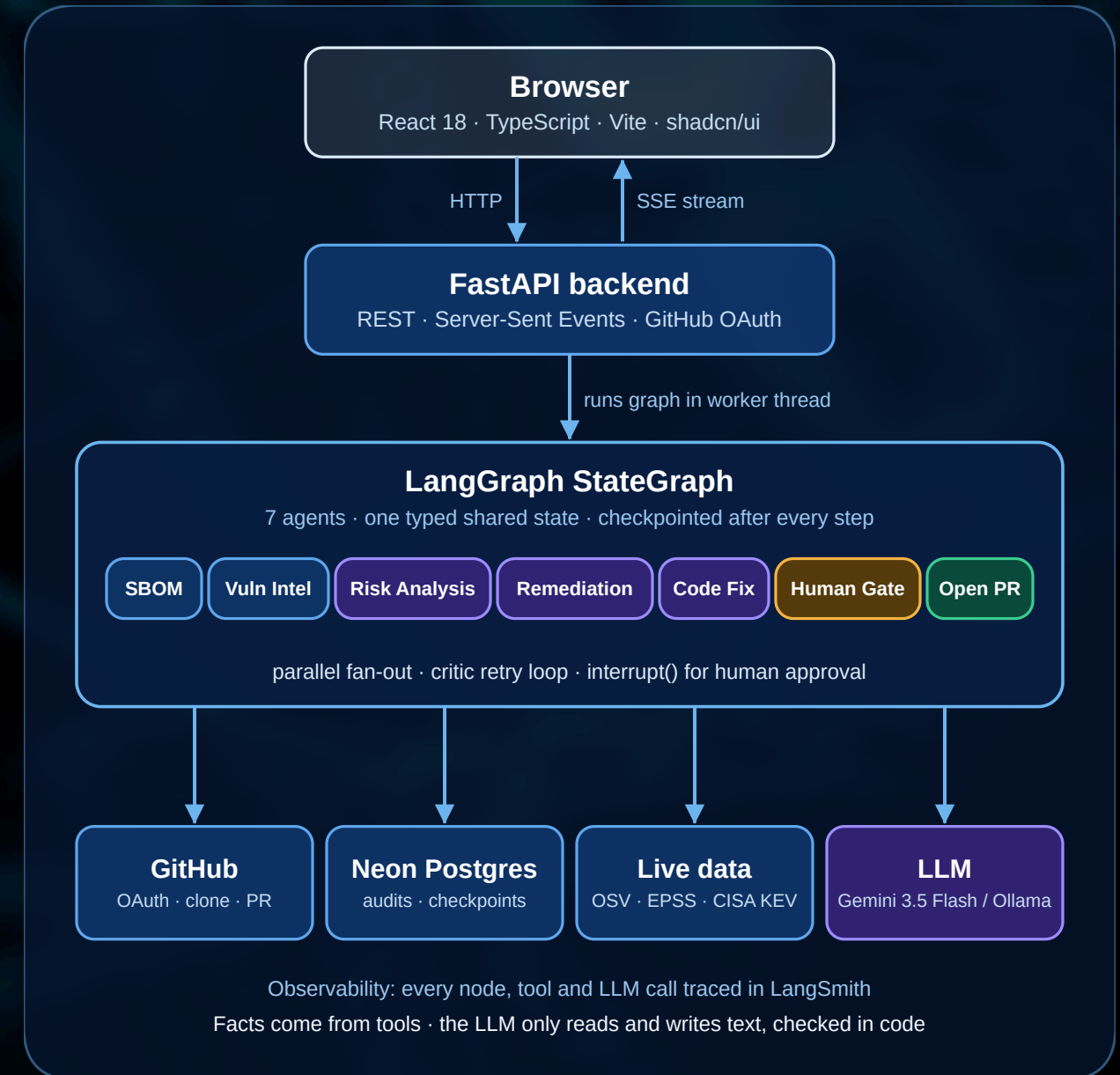
SYSTEM ARCHITECTURE

How components talk

The browser starts an audit over HTTP and follows it over one SSE stream. FastAPI runs the LangGraph pipeline in a worker thread; agents never call each other, they only read and write shared state. Postgres stores audits, cache and graph checkpoints.

Trust & safety by design

- Open PR is the only writer to GitHub, behind the human gate.
- No automatic major-version bumps.
- GitHub token stays server-side, encrypted at rest.
- Each repo installs in its own isolated virtualenv.



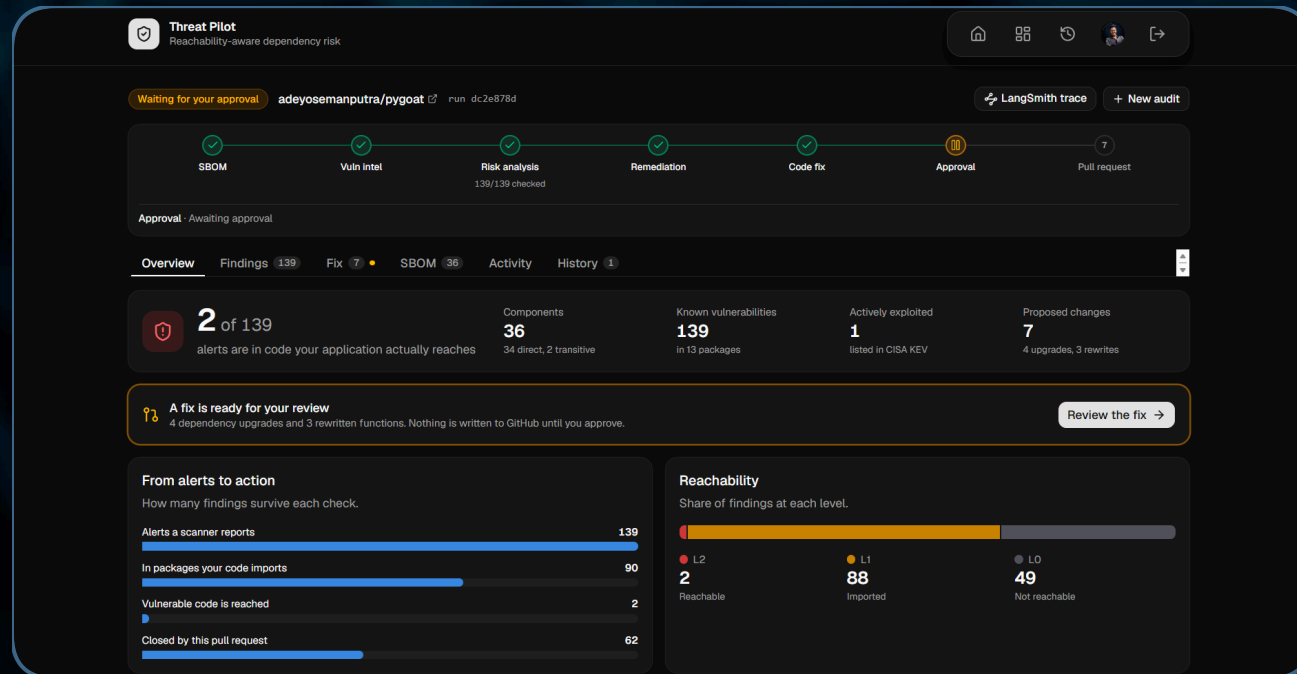
Home

Problem

Solution

Architecture

Impact



Prototype status

- Working end to end on real public repos.
- 55 backend tests pass with no network.
- Verified live against OSV, EPSS and CISA KEV.
- Real pull requests opened through GitHub.

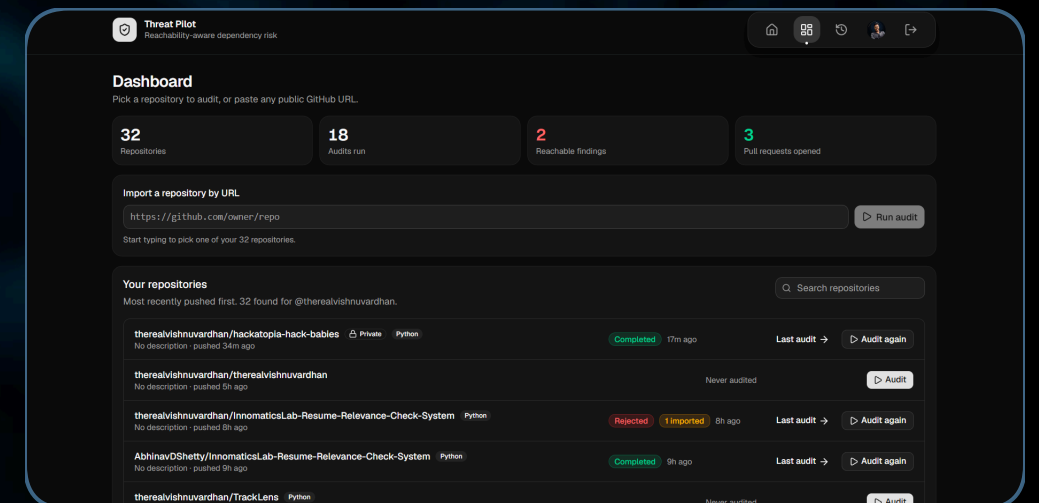
24-hour build strategy

- **0–2h** freeze state schema, stub all agents.
- **2–10h** real data, AST scanner, SSE UI.
- **10–16h** critic loop, interrupt/resume, integrate.
- **16–24h** polish, warm cache, rehearse.

BUILT & VERIFIED

Challenges → mitigation

- **LLM invents names** → validated against package source.
- **Venue network fails** → DB cache and OFFLINE mode.
- **Bad fix** → verifier loop, dry-run resolve.
- **Restart mid-approval** → Postgres checkpoints.





FUTURE SCOPE

GitHub Action

Run the graph on every pull request and comment the risk change it introduces.

Safer verification

Sandboxed installs in a container or microVM, and the project's own tests run against the fix.

Scale out

Job queue and workers on the checkpointed graph; organisations, roles and rate limits.

Blockchain trust layer

Sign each SBOM and audit result and anchor the hash on-chain, so anyone can verify what was audited and when.

More ecosystems

npm and Java through three adapters each: installer, tree-sitter parser, OSV ecosystem name.

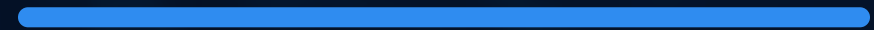
Smarter AI triage

LLM-assisted exploitability notes for L2 findings, still checked against evidence.

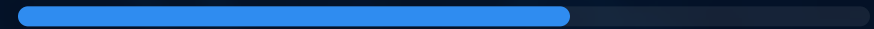
From alerts to action

Live audit of a public repo (pygoat), 36 components

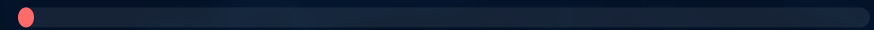
Alerts a scanner reports **139**



In packages your code imports **90**



Vulnerable code is reached **2**



Closed by one verified PR **62**



Key takeaway

Scanners tell you the vulnerable code is on your disk. Threat Pilot tells you whether your program runs it, and hands you the fix.

Three strongest benefits

- **Focus** 139 alerts become 2 that need action today.
- **Trust** file:line evidence, a verified fix and a human gate.
- **Speed** the pull request is drafted, checked and ready.

Vision

Every dependency alert arrives with proof and a reviewed fix: supply-chain security teams actually act on.

CONCLUSION



Home

Problem

Solution

Architecture

Impact



THANK YOU

🛡️ Threat Pilot · Agentic SBOM Risk Auditor
🔗 github.com/therealvishnuvardhan
📅 Hackatopia 2K26 · CB-02 · Questions welcome